

Software Testing Automation Report

21k3919 Zeeshan Mustafa

21k3807 Arqam

21k Kumail Raza

Project Name: Automation Testing for Ecommerce Website (Avantageonlineshopping.com) & Toucher Apk Mobile Application

Testing Framework: Selenium WebDriver, C# (.NET Framework), Visual Studio Code 2022

Introduction

In this project, we aim to automate the testing of the *Avantageonlineshopping.com* website, chosen due to its low security and well-defined end-to-end testing scenarios. The website is used for practicing automation tests involving registration, login, add-to-cart functionality, checkout processes, and account management.

Additionally, we have implemented automation testing for a mobile game application called *Toucher*, which is still in the development phase. In this case, we focus on testing core features such as login, adding scores, and adding friends. The game is being developed using Java, and we are using Allure for reporting in both the website and mobile game modules.

Test Strategy and Approach

We have followed the **Page Object Model (POM)** approach for structuring the automation tests. This ensures code reusability, maintainability, and a clear separation of concerns.

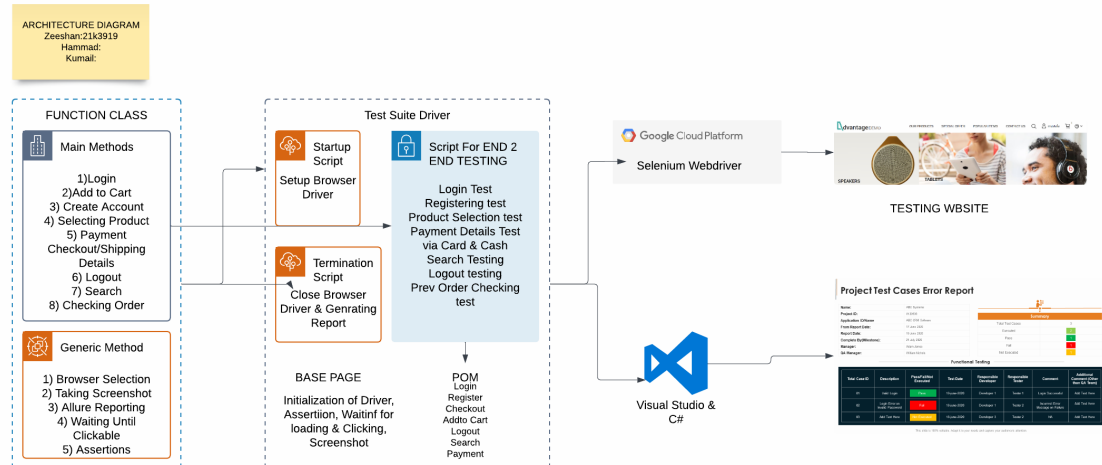
BasePage: Contains all the generic functions common across different pages, such as clicking elements, waiting for elements, inputting data, and handling dropdowns.

Page Objects: Each feature of the website and the mobile game, such as login, registration, cart, and adding scores/friends, is represented by a separate class with page-specific locators and methods.

System Architecture

The system architecture includes the following components:

- **Test Execution Layer:** The main layer where test cases are executed. It uses the Page Objects to interact with the UI and validate functionality.
- **Page Object Layer:** Contains individual classes for each page or feature, like LoginPage, RegisterAccount, AddToCart, etc. Each class has locators and actions specific to the page it represents.



Test Cases

The test cases cover various scenarios such as user registration, login, invalid login, adding to cart, checkout (with SafePay and MasterCard), signing out, and account deletion. The following are the key test cases executed:

- **Test Register:** Verifies the user registration functionality.
- **Test Login:** Verifies that the user can successfully log in with valid credentials.
- **Test Login Invalid:** Verifies that the system handles invalid login attempts correctly.
- **Test Add to Cart:** Verifies the add-to-cart functionality.
- **Test Checkout:** Verifies the checkout process with different payment methods (SafePay and MasterCard).
- **Test Add Score:** Verifies that the score is correctly added in the *Toucher* game.
- **Test Add Friends:** Verifies that users can add friends in the *Toucher* game.
- **Test Signout:** Verifies the sign-out functionality.
- **Test Delete Account:** Verifies the account deletion functionality.

Page Object Model (POM) Approach

The Page Object Model (POM) approach is used to separate the code that interacts with the user interface from the test logic. In this approach, we have created a Base Page that contains all generic functions for interacting with the website elements. Additionally, we have created specific page classes for each feature or page of the website. These page classes contain the locators, methods, and validation for that particular page.

The system architecture consists of the following components:

- BasePage: Contains generic functions for interacting with elements.
- Page Classes: Each feature or page is represented by a separate class (e.g., LoginPage, RegisterAccount, AddToCart).
- Test Execution: The TestExecution class handles the initialization of tests, execution of specific test cases, and validation.

Test Execution Code

Below is the code for TestExecution.cs and BasePage.cs.

TestExecution.cs

```
using System;
using EcommerceWebsite.POM;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using OpenQA.Selenium;
using System.Configuration;
using Allure.NUnit.Attributes;
using Allure.Net.Commons;
namespace EcommerceWebsite
{
    [TestClass]
    public class TestExecution
    {
        public static IWebDriver driver;
        public LoginPage loginPage;
        public RegisterAccount RegisterAccount;
        private AddToCart addToCartPage;
        private Checkout checkout;
        private MenuFunction menuFunction;

        [TestInitialize]
        public void Setup()
        {
            // Initialize the driver
            BasePage.SeleniumInit("Chrome");
            driver = BasePage.driver;
            loginPage = new LoginPage();
            RegisterAccount = new RegisterAccount();
            addToCartPage = new AddToCart();
            checkout = new Checkout();
        }
    }
}
```

```

        menuFunction = new MenuFunction();
    }

[TestMethod]
[TestCategory("Registration")]
[TestPriority(1)]
[AllureDescription("Test to verify user registration functionality")]
public void TestRegister_TC001()
{
    // Test data
    string username = "zeeshaniya";
    string email = "zeeshan@example.com";
    string password = "Zeeshan@123";
    string confirmPassword = "Zeeshan@123";
    string firstName = "Zeeshan";
    string lastName = "Mustafa";
    string phone = "1234567890";
    string country = "Pakistan";
    string city = "Karachi";
    string address = "123 Main Street";
    string state = "Sindh";
    string postalCode = "74000";
    bool receiveOffers = true;

    string Url = "https://advantageonlineshopping.com/#/";

    RegisterAccount.Register(Url, username, email, password, confirmPassword,
                             firstName, lastName, phone, country, city,
                             address, state, postalCode, receiveOffers);
}

[TestMethod]
[TestPriority(2)]
[AllureDescription("Test to verify user login functionality")]
public void TestLogin_TC002()
{
    string username = "zeeshaniya";
    string password = "Zeeshan@123";
    string Url = "https://advantageonlineshopping.com/#/";
    loginPage.Login(Url, username, password);
}

[TestMethod]

```

```

[TestPriority(3)]
[AllureDescription("Test to verify invalid login")]
public void TestLoginInvalid_TC003()
{
    string username = "kumail";
    string password = "kusdkjn";
    string Url = "https://advantageonlineshopping.com/#/";

    loginPage.Login(Url, username, password);

    try
    {
        string error = BasePage.driver.FindElement(By.Id("signInResultMessage")).Text;
        Assert.AreEqual(error, "OR");
    }
    catch (NoSuchElementException)
    {
        Assert.Fail("Login Successful");
    }
}

[TestMethod]
[AllureDescription("Test to verify Add to Cart functionality")]
public void TestAddToCart_TC004()
{
    string username = "zeeshaniya";
    string password = "Zeeshan@123";
    string Url = "https://advantageonlineshopping.com/#/";
    loginPage.Login(Url, username, password);
    addToCartPage.addToCart();
}
// Other test methods ...
[TestCleanup]
public void TearDown()
{
    BasePage.driver.Close();
}
}
}

```

BasePage.cs

```

using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
using OpenQA.Selenium.Support.Extensions;
using OpenQA.Selenium.Support.UI;
using SeleniumExtras.WaitHelpers;
using System;

namespace EcommerceWebsite
{
    public class BasePage
    {
        public static IWebDriver driver;

        public static void SeleniumInit(string browser)
        {
            if (browser == "Chrome")
            {
                var chromeOptions = new ChromeOptions();
                chromeOptions.AddArguments("--start-maximized");
                chromeOptions.AddArguments("--force-device-scale-factor=1.1");
                driver = new ChromeDriver(chromeOptions);
            }
        }

        private IWebElement WaitUntilElementIsClickable(By by, int timeoutInSeconds = 10)
        {
            WebDriverWait wait = new WebDriverWait(driver,
            TimeSpan.FromSeconds(timeoutInSeconds));
            return wait.Until(ExpectedConditions.ElementToBeClickable(by));
        }

        private IWebElement WaitUntilElementIsVisible(By by, int timeoutInSeconds = 10)
        {
            WebDriverWait wait = new WebDriverWait(driver,
            TimeSpan.FromSeconds(timeoutInSeconds));
            return wait.Until(ExpectedConditions.ElementIsVisible(by));
        }

        private void WaitForLoaderToDisappear(int timeoutInSeconds = 10)
        {
            WebDriverWait wait = new WebDriverWait(driver,
            TimeSpan.FromSeconds(timeoutInSeconds));
            wait.Until(driver =>

```

```

    {
        var loader = driver.FindElement(By.ClassName("loader"));
        return !loader.Displayed;
    });
}

public void Write(By by, string data)
{
    WaitForLoaderToDisappear();
    IWebElement element = WaitUntilElementIsVisible(by);
    element.SendKeys(data);
}

public void Click(By by)
{
    WaitForLoaderToDisappear();
    IWebElement element = WaitUntilElementIsClickable(by);
    element.Click();
}

public void SelectDropdownByText(By by, string text)
{
    WaitForLoaderToDisappear();
    IWebElement dropdownElement = WaitUntilElementIsClickable(by);
    SelectElement select = new SelectElement(dropdownElement);
    select.SelectByText(text);
}

public void OpenURL(string url)
{
    driver.Url = url;
}

public void ClickUsingJavaScript(By by)
{
    WaitForLoaderToDisappear();

    // Use JavaScript to click on the element
    IWebElement element = driver.FindElement(by);
    ((IJavaScriptExecutor)driver).ExecuteScript("arguments[0].click();", element);
}
}

```

}

Conclusion

The Selenium WebDriver automation testing for *Avantageonlineshopping.com* is structured using the Page Object Model (POM) approach. The test execution code interacts with the POM classes, ensuring the separation of concerns and maintaining clean and maintainable code. The automated tests verify critical functionalities such as registration, login, add-to-cart, checkout, and account management for the website. For the mobile game *Toucher*, we also ensured robust testing for login, score tracking, and adding friends using the same methodology and Allure for reporting. This approach ensures that both the website and mobile app maintain high-quality functionality.